

RELATÓRIO TÉCNICO DE MODULARIZAÇÃO

Disciplina: Algoritmos e Programação para Engenharia
Contexto de Aplicação: Engenharia Elétrica (Estação de Monitoramento de Tensão de Rede)
Foco do Estudo: Análise de Qualidade Estrutural e Robustez de Software via LLMs

Parte 1: Pseudo-código da Solução Autônoma

O algoritmo abaixo foi projetado de forma compacta para estruturar a lógica essencial do sistema de monitoramento de tensão (com base no valor nominal de 220V), mantendo a integridade das variáveis essenciais exigidas para a avaliação de qualidade.

```
ALGORITMO Monitoramento_Tensao_Rede

FUNÇÃO validar(v)
    MIN = 0; MAX = 500
    SE v é número E v >= MIN E v <= MAX ENTÃO RETORNAR Verdadeiro SENÃO RETORNAR Falso
FIM_FUNÇÃO

FUNÇÃO classificar(v)
    NOMINAL = 220; ALERTA = 0.10; CRITICO = 0.20
    desvio = ABS(v - NOMINAL) / NOMINAL

    SE desvio > CRITICO ENTÃO RETORNAR "Crítico"
    SE desvio > ALERTA ENTÃO RETORNAR "Alerta"
    RETORNAR "Normal"
FIM_FUNÇÃO

FUNÇÃO processar_dados(lista_bruta)
    lista_validos = []; descartados = 0; criticos = 0; seq = 0; maior_seq = 0

    // 1. Validação e Filtragem
    PARA CADA v EM lista_bruta FAÇA
        SE validar(v) ENTÃO Adicionar v em lista_validos SENÃO descartados = descartados +
1
    FIM_PARA

    // 2. Classificação e Sequência Crítica
    PARA CADA v EM lista_validos FAÇA
        status = classificar(v)
        Exibir "Valor: ", v, "V -> ", status

        SE status == "Crítico" ENTÃO
            criticos = criticos + 1; seq = seq + 1
            SE seq > maior_seq ENTÃO maior_seq = seq
        SENÃO
            seq = 0
    FIM_SE
FIM_PARA

    // 3. Relatório Final e Estatísticas
```

```

    Exibir "Válidos: ", COMPRIMENTO(lista_validos), " | Descartados: ", descartados
    Exibir "Média: ", Média(lista_validos), " | Mínimo: ", Mínimo(lista_validos), " |
Máximo: ", Máximo(lista_validos)
    Exibir "Desvio Padrão: ", Desvio_Padrão(lista_validos)
    Exibir "Total Críticos: ", criticos, " | Maior Sequência Consecutiva: ", maior_seq
FIM_FUNÇÃO

```

Parte 2: Implementações em Python Geradas pelas LLMs

Ferramenta A: Microsoft Copilot

```

import statistics

def validar_medicao(medicao):
    try:
        valor = float(medicao)
        if 0 <= valor <= 500:
            return valor
        else:
            return None
    except (ValueError, TypeError):
        return None

def classificar_medicao(valor, nominal=220):
    desvio_percentual = abs((valor - nominal) / nominal) * 100
    if desvio_percentual <= 10:
        return "Normal"
    elif desvio_percentual <= 20:
        return "Alerta"
    else:
        return "Crítico"

def detectar_eventos_criticos(classificacoes):
    total_criticos = classificacoes.count("Crítico")
    maior_sequencia = 0
    sequencia_atual = 0
    for c in classificacoes:
        if c == "Crítico":
            sequencia_atual += 1
            maior_sequencia = max(maior_sequencia, sequencia_atual)
        else:
            sequencia_atual = 0
    return total_criticos, maior_sequencia

def calcular_estatisticas(valores_validos, total_descartados):
    if not valores_validos:
        return None
    media = statistics.mean(valores_validos)
    minimo = min(valores_validos)
    maximo = max(valores_validos)
    desvio_padrao = statistics.pstdev(valores_validos)
    total_validos = len(valores_validos)
    return {
        "Média": media, "Mínimo": minimo, "Máximo": maximo,
        "Desvio Padrão": desvio_padrao, "Válidos": total_validos, "Descartados":

```

```

total_descartados
}

def main(medicoes_brutas):
    valores_validos = []
    classificacoes = []
    descartados = 0
    for m in medicoes_brutas:
        valor = validar_medicao(m)
        if valor is not None:
            valores_validos.append(valor)
            classificacoes.append(classificar_medicao(valor))
        else:
            descartados += 1
    print("Classificação das medições válidas:")
    for valor, classe in zip(valores_validos, classificacoes):
        print(f" {valor:.2f} V -> {classe}")
    total_criticos, maior_sequencia = detectar_eventos_criticos(classificacoes)
    estatisticas = calcular_estatisticas(valores_validos, descartados)
    print("
--- Relatório Final ---")
    if estatisticas:
        for chave, valor in estatisticas.items():
            print(f"{chave}: {valor}")
    print(f"Total de medições críticas: {total_criticos}")
    print(f"Maior sequência consecutiva de críticas: {maior_sequencia}")

if __name__ == "__main__":
    dados_teste = [220, 215, 250, 300, "erro_sensor", 180, 220, 280, 290, 150]
    main(dados_teste)

```

Ferramenta B: OpenAI ChatGPT

```

import math

def validar_medicao(valor):
    if isinstance(valor, (int, float)) and 0 <= valor <= 500:
        return True
    return False

def classificar_tensao(tensao, nominal=220):
    desvio = abs(tensao - nominal) / nominal
    if desvio > 0.20:
        return "Crítico"
    elif desvio > 0.10:
        return "Alerta"
    else:
        return "Normal"

def calcular_media(valores):
    return sum(valores) / len(valores)

def calcular_desvio_padrao(valores):
    media = calcular_media(valores)
    soma_quadrados = 0
    for valor in valores:

```

```

        soma_quadrados += (valor - media) ** 2
    variancia = soma_quadrados / len(valores)
    return math.sqrt(variancia)

def calcular_estatisticas(valores):
    return {
        "media": calcular_media(valores), "minimo": min(valores),
        "maximo": max(valores), "desvio_padrao": calcular_desvio_padrao(valores)
    }

def detectar_eventos_criticos(classificacoes):
    total_criticos = 0; sequencia_atual = 0; maior_sequencia = 0
    for status in classificacoes:
        if status == "Crítico":
            total_criticos += 1; sequencia_atual += 1
            if sequencia_atual > maior_sequencia:
                maior_sequencia = sequencia_atual
        else:
            sequencia_atual = 0
    return total_criticos, maior_sequencia

def processar_medicoes(lista_bruta):
    medidas_validas = []; classificacoes = []; descartadas = 0
    print("
CLASSIFICAÇÃO DAS MEDIÇÕES")
    for valor in lista_bruta:
        if validar_medicao(valor):
            medidas_validas.append(valor)
            status = classificar_tensao(valor)
            classificacoes.append(status)
            print(f"{valor:>6} V -> {status}")
        else:
            descartadas += 1
            print(f"{str(valor):>6} -> DESCARTADO")
    return medidas_validas, classificacoes, descartadas

def gerar_relatorio(validas, descartadas, classificacoes):
    estatisticas = calcular_estatisticas(validas)
    total_criticos, maior_sequencia = detectar_eventos_criticos(classificacoes)
    print("
===== RELATÓRIO FINAL =====")
    print(f"Amostras válidas: {len(validas)} | Descartadas: {descartadas}")
    print(f"Média: {estatisticas['media']:.2f} V | Mínimo: {estatisticas['minimo']:.2f}
V")
    print(f"Máximo: {estatisticas['maximo']:.2f} V | Desvio padrão:
{estatisticas['desvio_padrao']:.2f} V")
    print(f"Total de críticos: {total_criticos} | Maior sequência: {maior_sequencia}")

def main(lista_bruta):
    validas, classificacoes, descartadas = processar_medicoes(lista_bruta)
    gerar_relatorio(validas, descartadas, classificacoes)

if __name__ == "__main__":
    medicoes_brutas = [220, 215, 250, 300, "erro_sensor", 180, 220, 280, 290, 150]
    main(medicoes_brutas)

```

Parte 3: Análise Comparativa e Avaliação de Engenharia

#	Critério de Modularização Analisado	Copilot	ChatGPT
1	Decomposição em funções (responsabilidade única)	2	2
2	Nomes descritivos de funções e variáveis	2	2
3	Ausência de duplicação de código (Conceito DRY)	2	2
4	Parâmetros e retornos bem definidos (sem usar variáveis globais)	2	2
5	Reaproveitamento lógico de funções	2	2
6	Existência de uma função principal organizando o fluxo	2	2
7	Legibilidade geral, blindagem e boas práticas de programação	2	1
-	Pontuação Final Obtida (Total Máximo: 14)	14	13

Análise Crítica e Diferencial de Robustez

A auditoria técnica aprofundada dos códigos revelou que, embora ambos os modelos tenham atingido excelente divisão modular, o **Microsoft Copilot obteve um desempenho superior em termos de engenharia de software e blindagem de código**. O diferencial crítico reside na arquitetura da função de validação de dados. O Copilot implementou uma estrutura resiliente de tratamento de exceções com blocos `try/except`, que tenta ativamente converter as entradas. Se uma medição válida for submetida acidentalmente como string (ex: "220"), o Copilot recupera o dado com sucesso; se uma falha severa ocorrer (ex: "erro_sensor"), o bloco captura o erro de forma elegante, impedindo que o sistema sofra um *crash* estrutural.

Em contrapartida, o ChatGPT adotou uma validação puramente teórica e frágil através do método `isinstance(valor, (int, float))`. Em cenários reais de instrumentação elétrica, esta escolha ignora dados em formato de string numérica legítima e deixa o sistema vulnerável caso tipos de dados inesperados burlam a entrada. Em relação ao pseudo-código original da Parte 1, ambas as IAs refinaram a arquitetura ao isolar as exibições em console (prints) e cálculos matemáticos secundários, melhorando a granularidade do projeto inicial.

REFLEXÃO DE ENGENHARIA

Em sistemas industriais reais, a modularização e a blindagem de código são vitais. O isolamento de funções permite realizar testes unitários cirúrgicos em bancadas antes da integração total do sistema. Uma validação robusta (como a do Copilot) garante que ruídos e falhas nos sensores de campo não derrubem a aplicação de monitoramento central. Adicionalmente, códigos altamente modulares otimizam a manutenção preventiva e permitem o reuso imediato de rotinas matemáticas e lógicas em projetos futuros de monitoramento de outras grandezas elétricas, poupando tempo e custo corporativo.